

Basic Concepts to Remember.

Integer. Max-value $\rightarrow$ It is the highest integer value of any integer.

Integer. Min-value $\rightarrow$ It is the lowest integer value of any integer.

When to use for or while loop? ①

$\rightarrow$ ~~for~~ while loop is used, when we don't know whether we have to increment the counter in first or anywhere in the code.

But when we know that the increment can be done in the ~~start~~ starting of the next execution.

[2] .

[1] .

[5-1] .

[1-2-5] .

[2-5-3] .

[1-2-5-3] .

$\rightarrow$ In this case, the condition becomes true in the next iteration.

Kadane's Algorithm.

Kadane's Algorithm is an efficient method used to find the maximum sum of a contiguous subarray.

* It is used in only 2D array

① Contiguous Subarray.

↳ It means the elements are next to each other in the original array.

↳ ~~without shift~~

Example $[3, -2, 5, -1]$

• $[3]$

• $[-2]$

• $[5]$

• $[1]$

• $[3, -2]$

• $[-2, 5, -1]$

• $[3, -2, 5]$

• $[3, -2, 5, -1]$

These are all contiguous. because they are made elements in order

Subarray

→ contiguous part of Array.

Example from $[1, 2, 3, 4]$

→ $[1, 2]$, $[2, 3, 4]$

Subset

→ It is a combination of element from the array. The order doesn't matter and you can flip any element.

Example from $[1, 2, 3]$

$[\]$, $[1]$, $[2]$, $[3]$, $[1, 2]$, $[1, 3]$, $[2, 3]$, $[1, 2, 3]$

- * It includes the empty set and the full set.
- * It can be contiguous or non-contiguous.

Subsequence

→ A Subsequence is formed by removing some elements, but keeping the original order.

- * The element must appear in the same left to right order as in the original array.

* It can be non-contiguous, but the order will be same.

* Kadane's is usually used in solving maximum sub array sum of array

use case of Kadane's algorithm

* use when you are given.

→ An array of integers (can be positive or negative)

And asked to find the maximum sum of a contiguous subarray.

Time complexity $\rightarrow O(n)$

Space complexity $\rightarrow O(1)$

Kadane's intuition.

- Should I extend the current subarray.
- Should I start fresh from the current element.

Real idea behind it:

- Going through the array from start to end.
- Each step we are keeping track of.
 - Best sum I can get ending right here.
 - What is the overall best sum I have seen anywhere so far.

we have two choices at every element.

- ① continue the subarray, you are building.
- ② start a new subarray from the current element



Trick.

i) adding current number makes thing worse (means negative). just start fresh from the current number.

otherwise, keep going and add it to your current running sum.

Key Idea.

- ① Traverse through the array, and keep track of two variables.
- ② max_ending_here → Represents the maximum sum of subarray ending at current position.
- ③ Max_so_far → Represents the maximum sum of subarray ending at current position.

* keep track of current sum and max sum

we have two main to avoid out and in

②

① constant the subarray from the array
 ② Start a first/2
 elements

[3, -2, 5, -1]

① one variable to check sum of whole array

② one variable to check the highest sum we have seen so far

③ check if the current sum becomes zero, then reset its value to 0

④ check if the current sum is greater than max sum we have seen then make the max sum equal to the value of current sum

①
 ②
 ③
 ④

In code. ① way.

```
int arr = {-2, -4}; // (0 > -2) // unique  
int current = arr[0]; // start with 1st element  
int max = arr[0];  
int n = arr.length;  
for (int i = 1; i < n; i++) {  
    if (current + arr[i] > arr[i]) {  
        current = current + arr[i];  
    } else {  
        current = arr[i];  
    }  
    if (current > max) {  
        max = current;  
    }  
}
```

② way.

```
int[] arr = {-2, -4};  
int n = arr.length;  
int currentSum = 0;  
int max = Integer.MIN_VALUE;  
for (int i = 0; i < n; i++) {  
    current = currentSum + arr[i];  
    if (current > max) {  
        max = current;  
    }  
}
```

